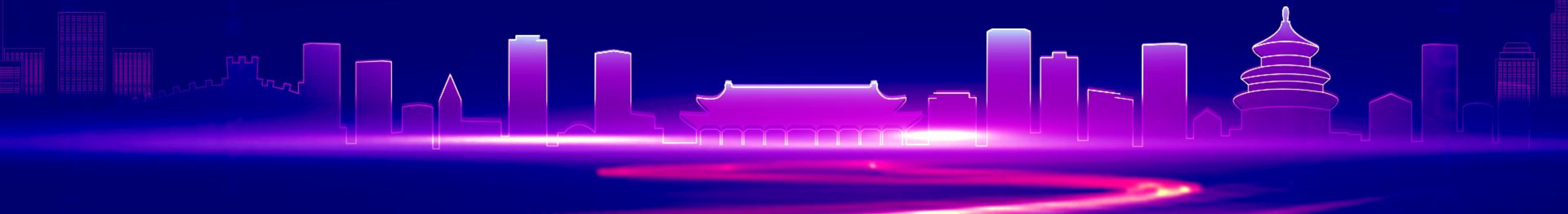
仓颉：一款全场景智能化应用编程语言

冯新宇
华为编程语言实验室



报告提纲

- 仓颉语言背景介绍
- 设计理念和技术特性
- 大模型下的编程语言思考
- 未来方向和规划

编程语言发展历史

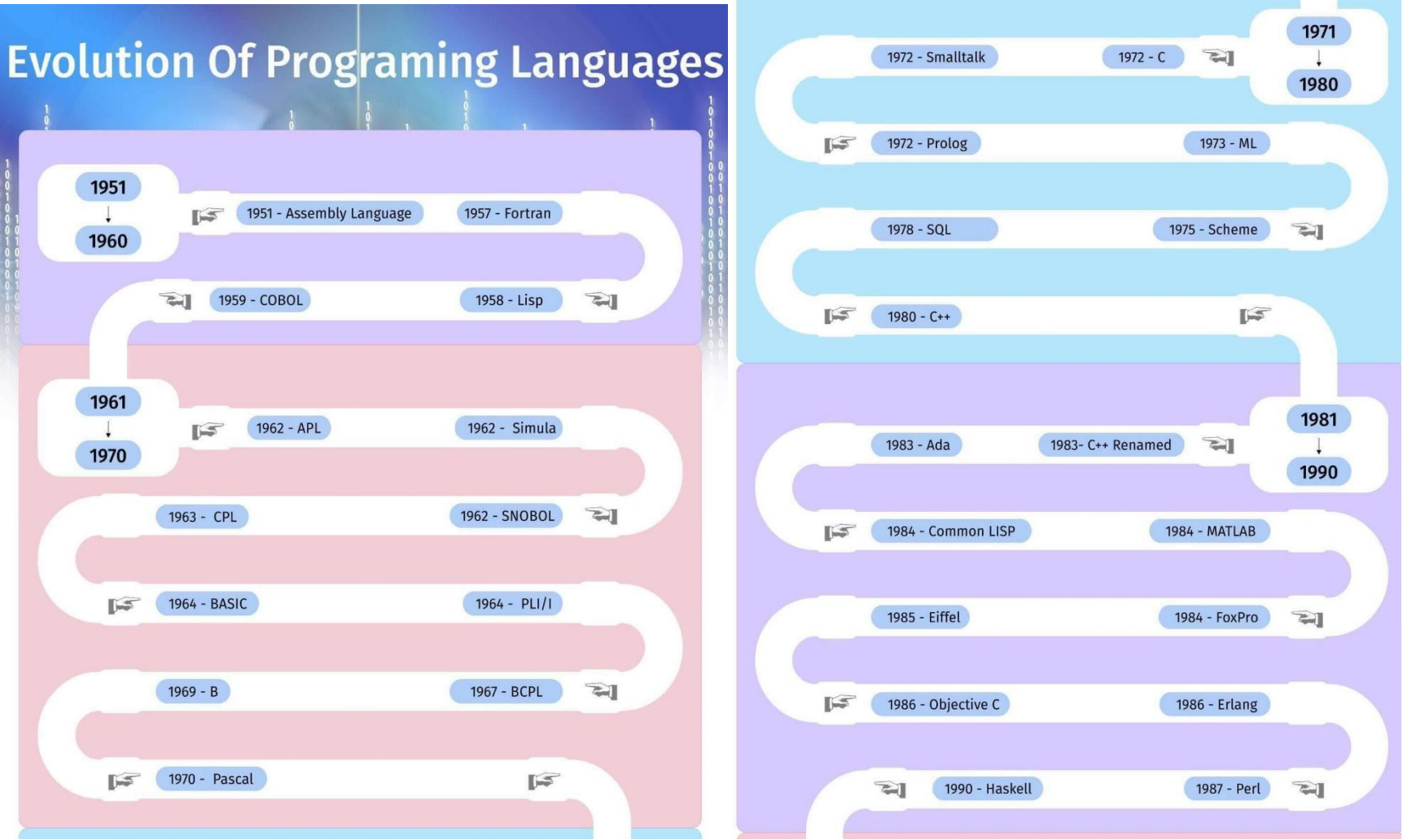


Image from www.technolush.com

为什么要新语言

解决特定的技术问题，实现特定的编程理念

产业/技术变革需要新的开发范式：往往聚焦特定领域

个人兴趣/学术界研究推广特定语言范式或特性

引导开发者，繁荣开发者生态

平台型企业主导，生态策略为主

微软：C#
苹果：Swift
Google: Go、Dart、(Kotlin)
…

为什么要仓颉语言

- 繁荣HarmonyOS应用生态，打造最适合HarmonyOS的应用开发语言

  提升开发体验和程序性能，为鸿蒙汇聚更多的应用开发者

  发挥系统优势，垂直整合，最大化的将系统的竞争力透传给开发者

- 能力辐射其他领域，支持全场景应用开发

- 迎接产业/技术变革

  智能化、空间化、一体化 …

  编程语言作为一个平台，有很大的技术演进与成长空间

仓颉语言的发展历程

2019

正式立项

仓颉语言团队成立

- 仓颉语言项目**正式通过立项**
- 与南大成立“**下一代程序设计创新实验室**”
- 与北大**胡振江教授团队**建立技术合作，开展元编程技术研究

2021

建立编程语言技术社区

语言原型构建

- 完成**原型构建**，性能竞争力显现
- 建立国内首个**编程语言技术社区，**启动“**内测**”
- 2021.10华为HDC**预告华为自研编程语言仓颉**

2022

仓颉语言首商用

内部应用试点

- **规范、工具链、标准库**核心能力基本具备
- **在数通S380**路由交换一体机**首商用**
- 华为**网管系统、服务器iBMC、云核业务软件**选用仓颉试点



2023

头部企业加入

外部应用试点

- 国内14家单位（工行、中移动...）首批加入**仓颉外部试点**
- 多家高校（北大、南大、北航、湖大...）参与**仓颉技术构建**
- 内测开发者**突破3000**，贡献代码突破**130万行**

2024+

仓颉语言发布

预览版发布

- 2024.6.21华为HDC大会**仓颉预览版发布**
- **工行、力扣** HarmonyOS NEXT版本选用仓颉开发

https://e.huawei.com/cn/videolist/enterprise/29ece690231a4705967dedb03916d97a

仓颉语言的应用案例

作为鸿蒙全场景智能应用编程语言，支撑工行、力扣等多款鸿蒙原生应用开发

力扣APP

力扣APP端到端支持仓颉全量开发，关键性能指标超越竞品语言（2024.7上架HarmonyOS NEXT应用市场）



力扣APP性能收益

- **“更快”的力扣**：冷启动性能提升3倍（3.32s -> 1.06s）
- **“更丝滑”的力扣**：在力扣中浏览多图的文章或题解页面等重载场景，仓颉版本丝滑流畅

工行APP

工行手机银行，关键模块“收支日历” 选用仓颉开发，解决复杂页面性能问题（2024.9上架HarmonyOS NEXT应用市场）



场景特征

- **页面复杂**：页面包括日历、日期选择滚轮等多个组件，嵌套及布局复杂
- **性能要求高**：收支明细数据加载及展示涉及较为复杂的数据解析及视图渲染工作，性能要求较高

仓颉支持能力

- **混合开发能力**，与鸿蒙ArkTS工程混合开发能力，支持同一个APP中使用仓颉及ArkTS语言
- **跨语言互操作能力**，支持仓颉与ArkTS、C语言间便捷完成数据共享、API调用

收益

- 页面顺滑，无卡顿

HUAWEI

作为应用编程语言，支撑金融、电信、政务等行业应用开发

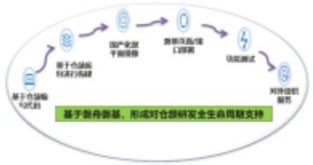
ICBC 中国工商银行

工行短信系统/微信系统



中国移动 China Mobile

磐基磐舟平台支持仓颉应用



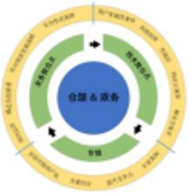
CSII科蓝

金融交易回溯系统



泛微 weaver

公文交换系统



中间件&三方库

BES 宝兰德 | PRIMETON 普元 | 东方通 TongTech | Apusic 金蝶天燕

Redis中间件 分布式追踪中间件 S3中间件 OBS中间件 分布式配置

Opengauss驱动 盘维数据库驱动 …

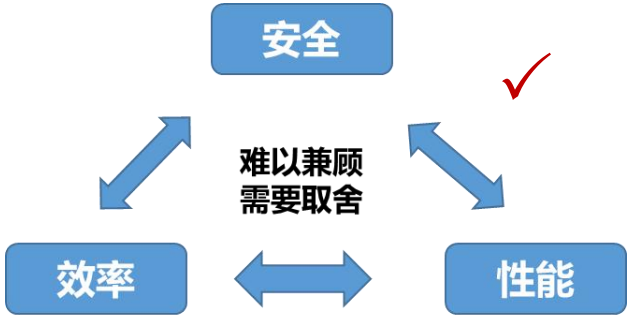
HUAWEI

报告提纲

- 仓颉语言背景介绍
- 设计理念和技术特性
- 大模型下的编程语言思考
- 未来方向和规划

整体理念和思路

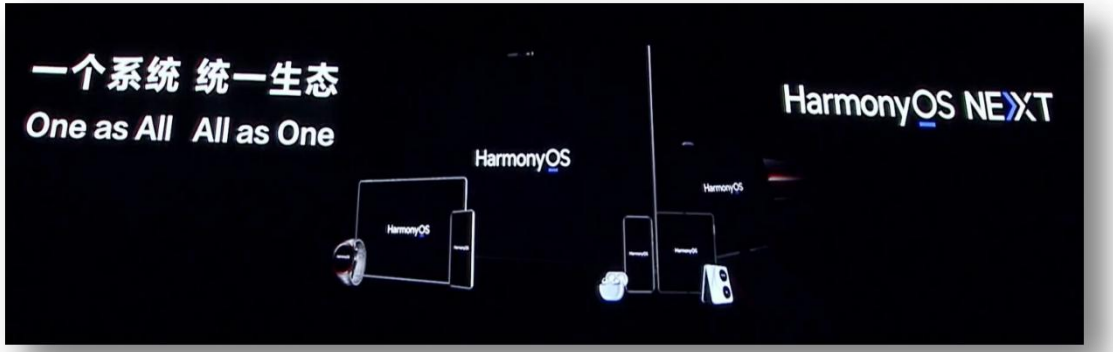
做好当代：全场景**应用**开发



迎接未来：

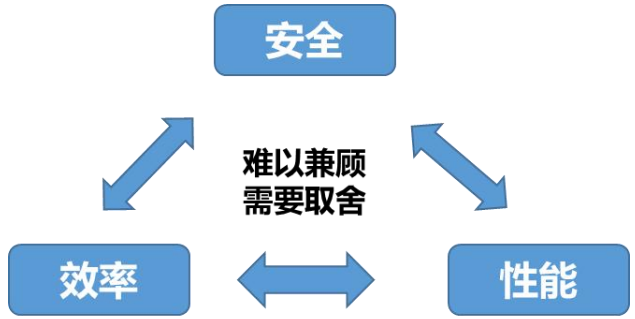
✓ 全场景 ✓ 智能化 …

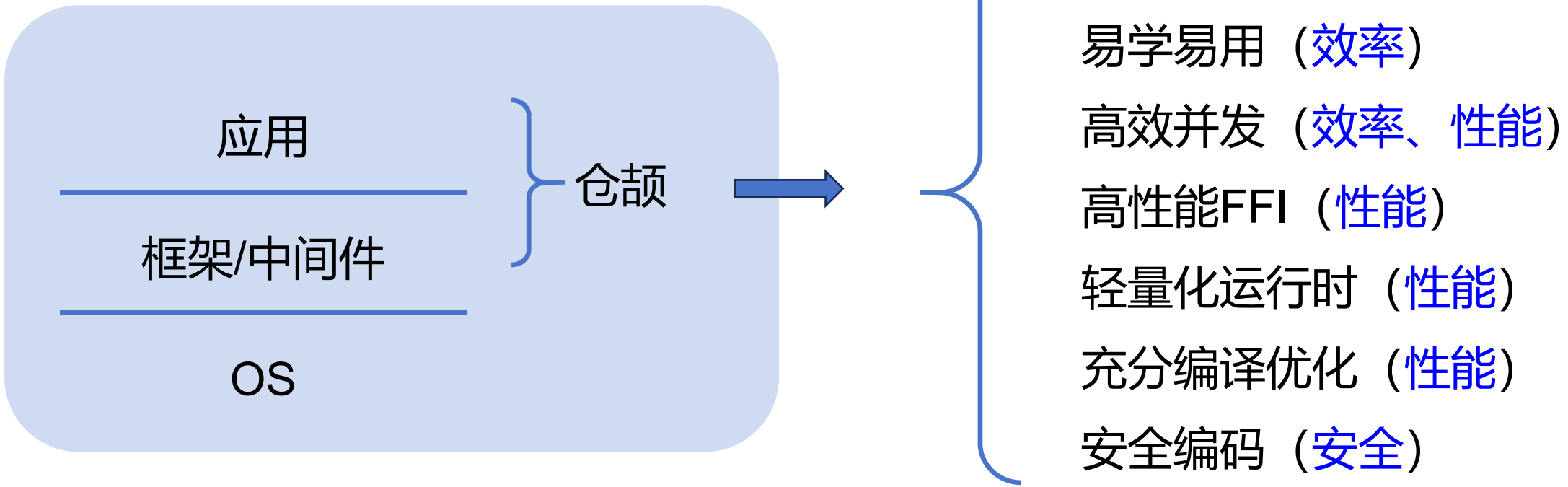
✓ 元编程，以支持良好的领域扩展



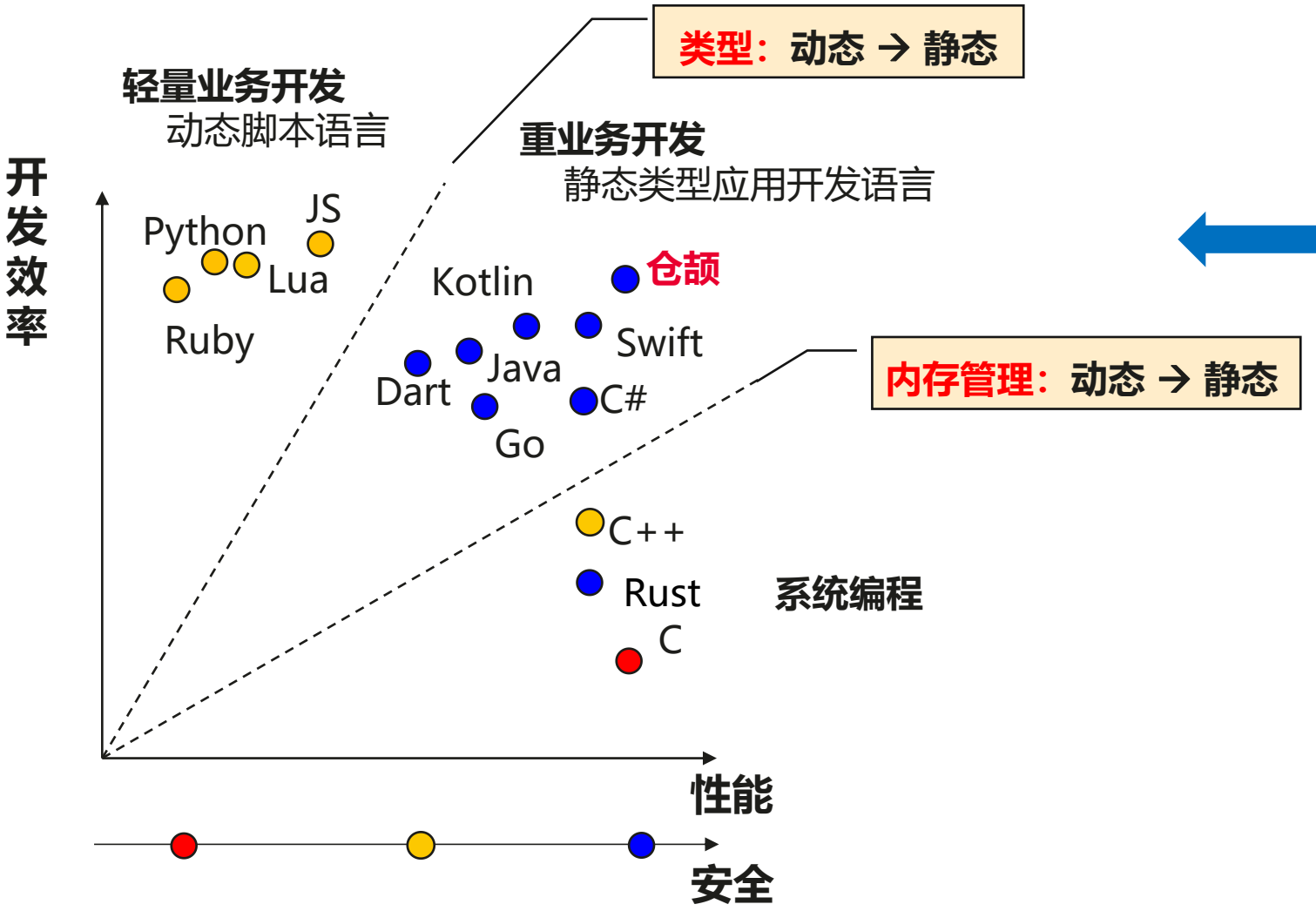


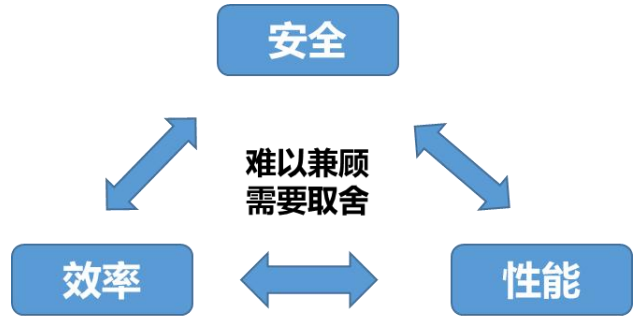
做好当代：全场景**应用**开发





仓颉的技术选型





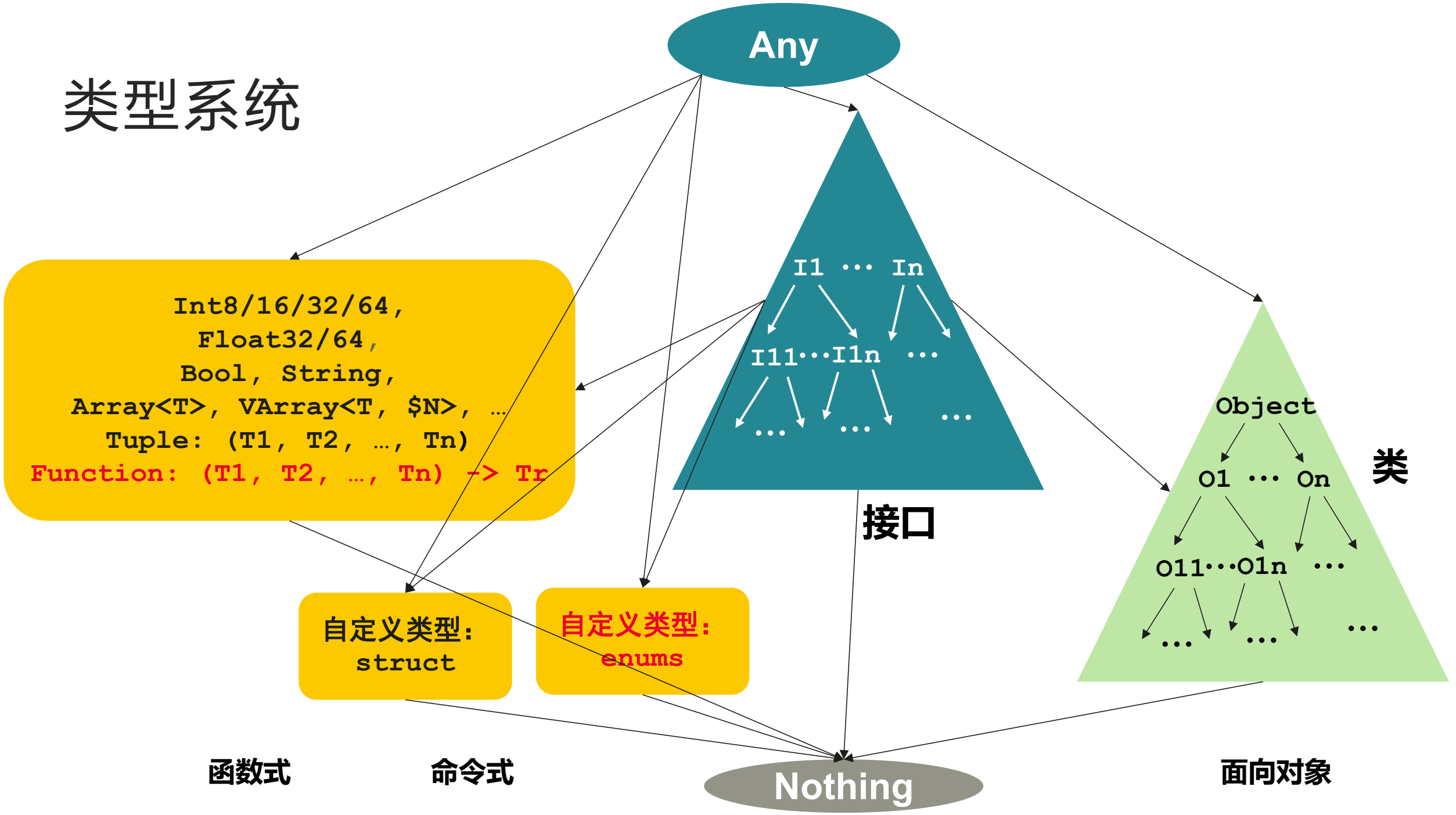
仓颉

自动内存管理（垃圾收集）

静态类型

注：此图仅为示意三类语言的划分，不追求各种语言在坐标中位置的准确性

类型系统


Any
Int8/16/32/64,
Float32/64,
Bool, String,
Array<T>, VArray<T, $N>, …
Tuple: (T1, T2, …, Tn)
Function: (T1, T2, …, Tn) -> Tr
I1 … In
I11…I1n …
接口
Object
O1 … On
O11…O1n …
类
自定义类型:
struct
自定义类型:
enums
函数式
命令式
Nothing
面向对象

主要语言特性

OO特性：接口、类、继承、子类型、属性

函数式特性：高阶函数、代数数据类型、模式匹配、不可变变量等

函数重载、操作符重载

泛型

类型扩展

并发

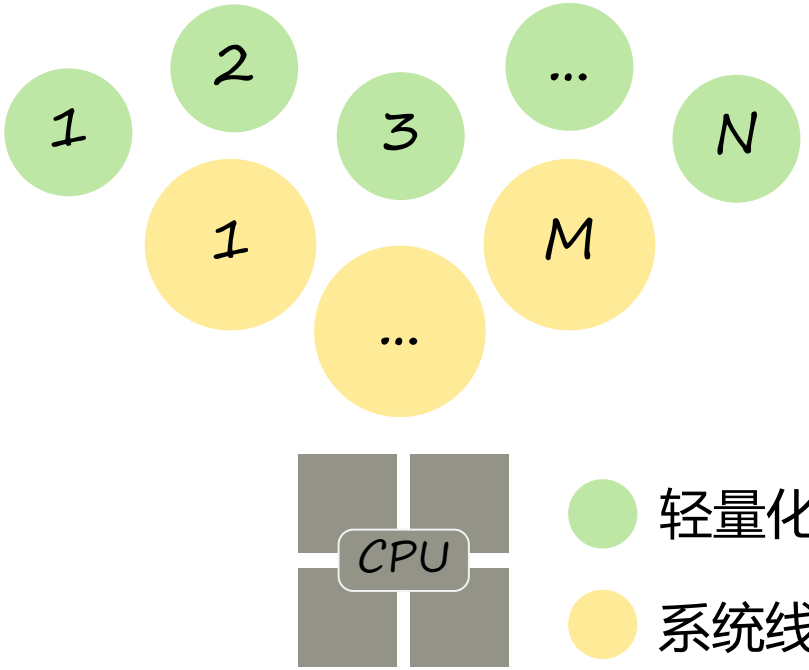
宏

反射/注解

…

轻量化用户态线程（a.k.a 协程）

M:N 轻量化线程模型




```
func concurrency() {
    let f = spawn {
        for (i in 0..100) {
            println("Hello from another thread")
            sleep(Duration.second * 1)
        }
    }

    f.get()
}
```

并发数据对象

并发对象库 —— 设计和实现

- **ConcurrentHashMap**
  - 线程安全的 Map 操作
    - 键值对：插入、删除、查询、替换 ...
  - 无锁实现（算法基于 ☞ ）
- **NonBlockingQueue**
  - 无界非阻塞的并发安全队列
    - 入队、出队、队首元素查询 ...
  - 无锁实现（核心思想 ☞ ）
- **BlockingQueue**
  - 扩充 NonBlockingQueue 算法实现：
    - 用户自定义队列容量上界
    - 出队/出队失败，线程阻塞
    - 出队/出队成功，阻塞线程唤醒

【Liu'et al. PODC'14】

Dynamic-Sized Nonblocking Hash Tables*

Yujie Liu
Lehigh University
lyj@lehigh.edu

Kunlong Zhang
Tianjin University
zhangkl@tju.edu.cn

Michael Spear
Lehigh University
spear@cse.lehigh.edu

ABSTRACT

This paper presents nonblocking hash table algorithms that support resizing in both directions: shrinking and growing. The heart of the table is a fixeable set abstraction, which greatly simplifies the task of moving elements among buckets during a resize. Furthermore, resize operation (or rehash) must be performed on the hash table to extend the size of the bucket array. However, resizing a hash table in the presence of concurrent operations in a nonblocking manner is a difficult problem. For example, Shalev and Shavit [17] state:



数组 + 链表 的实现方式

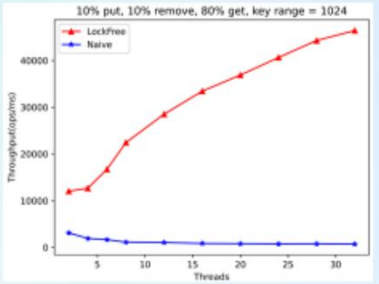
队列中元素

连续空间，高效 ☺

动态分配，支持无界 ☺

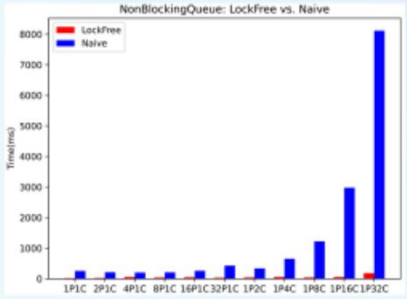
并发对象库 —— 性能测试

- **ConcurrentHashMap**
  - 比较对象：粗粒度所锁实现（互斥锁 + HashMap）
  - 测试结果：无锁 比 粗粒度锁 实现优势明显 ☞
    - 无锁实现：线程增加，性能提升！
    - 部分场景性能最大提升 60x
- **NonBlockingQueue**
  - 比较对象：粗粒度所锁实现（互斥锁 + 数组）
  - 测试结果：无锁 比 粗粒度锁 实现优势明显 ☞
    - 无锁实现：线程增加，性能不会有明显下降！
    - 部分场景性能最大提升 40x
- **后续任务：更完善的性能测试结果**
  - 并发数据结构对实际应用场景的性能提升
  - 并发数据结构：仓颉 vs. 其它语言（Java、C# ...）
  - ...



LockFree：无锁实现
Naive：粗粒度锁实现

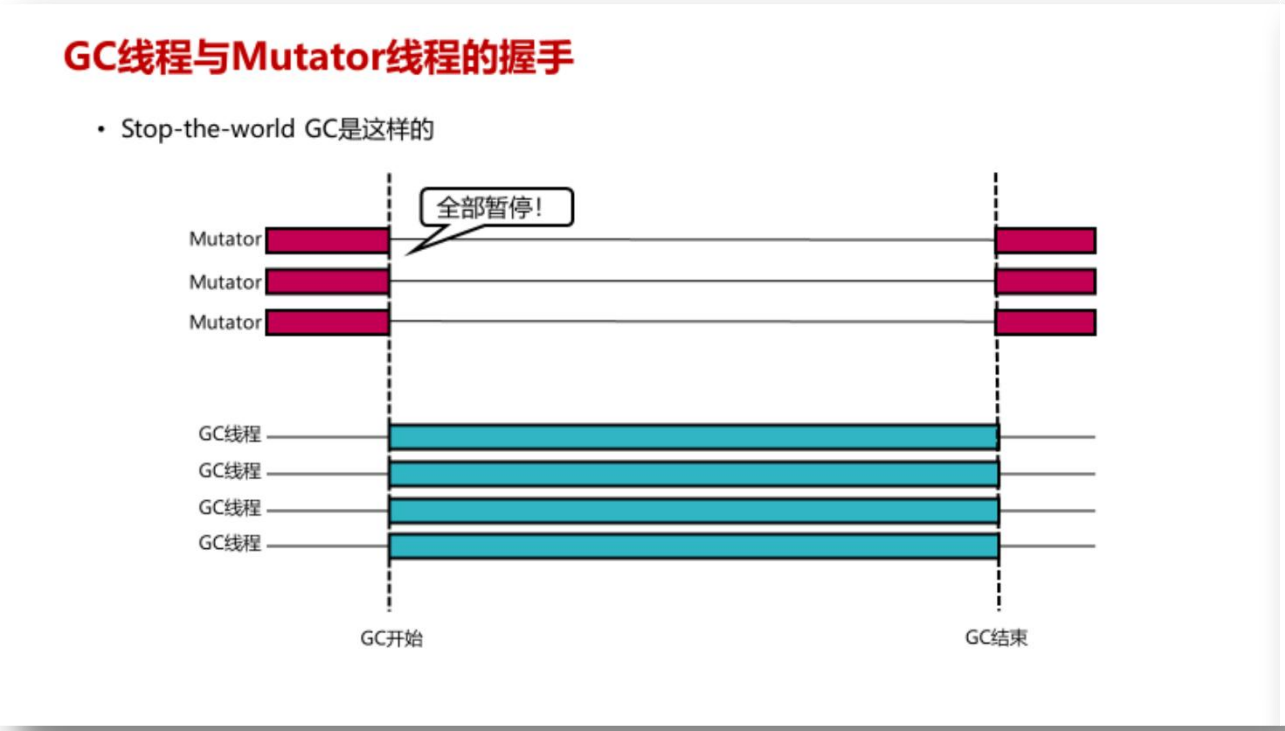
纵坐标：
单位时间完成操作数
越大性能越优！

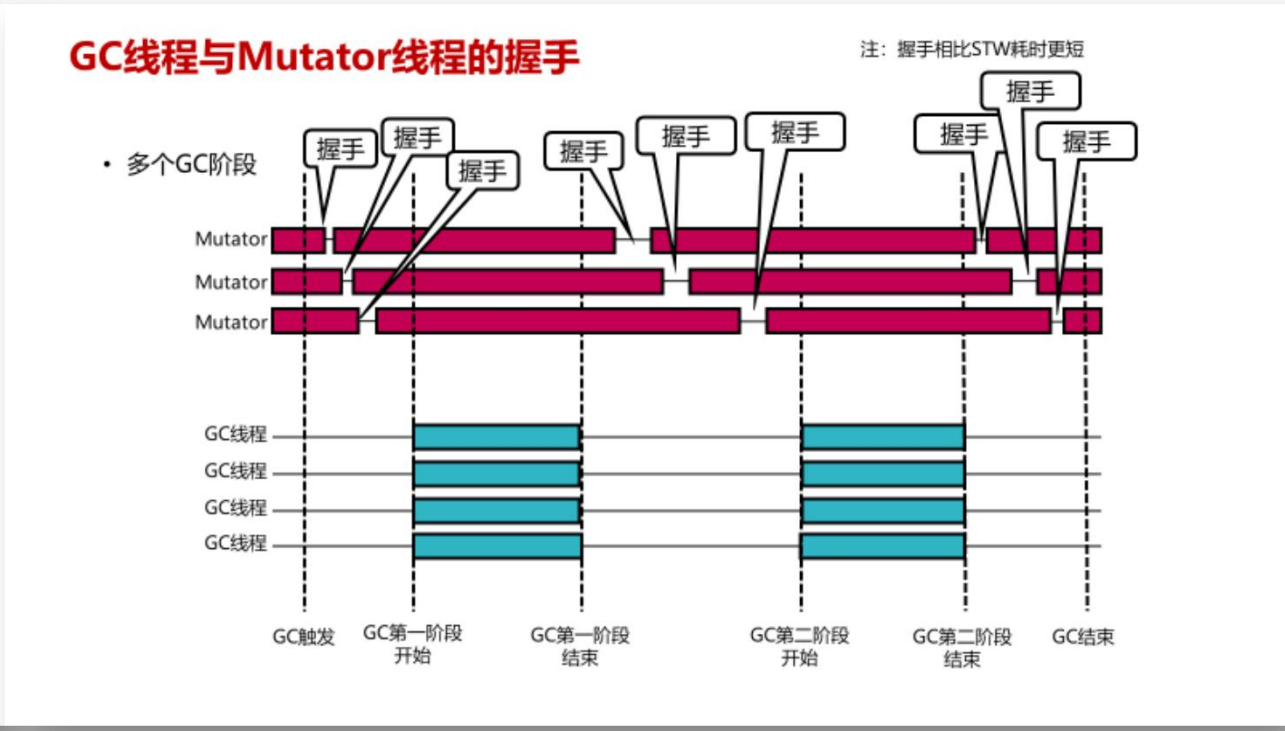


LockFree：无锁实现
Naive：粗粒度锁实现

纵坐标：
200000 个元素入队
和出队时间
越小性能越优！

并发GC

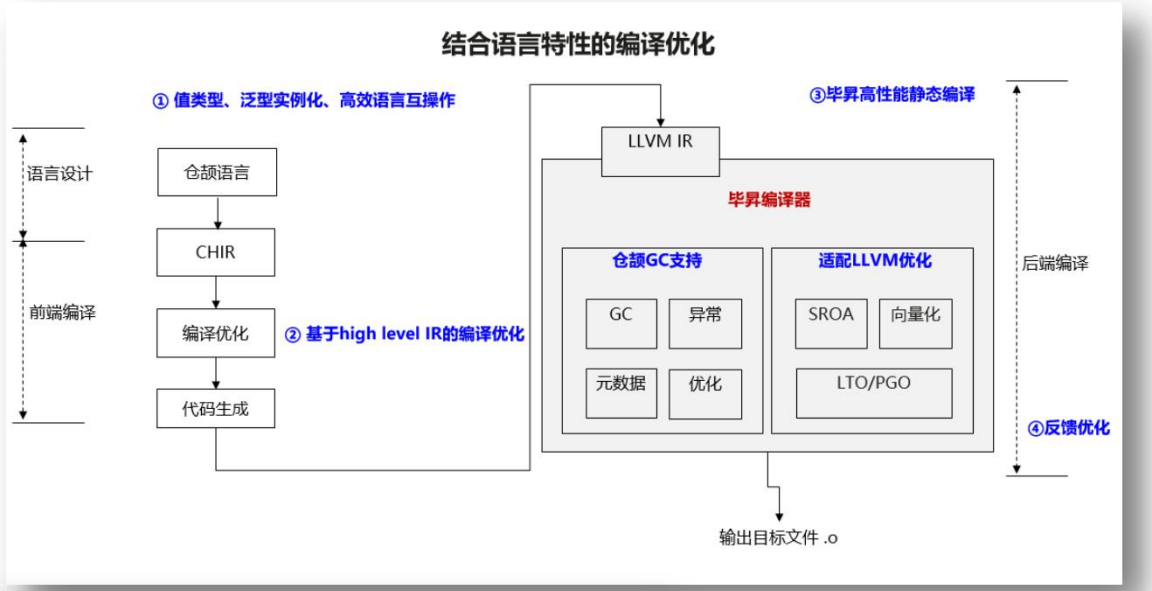




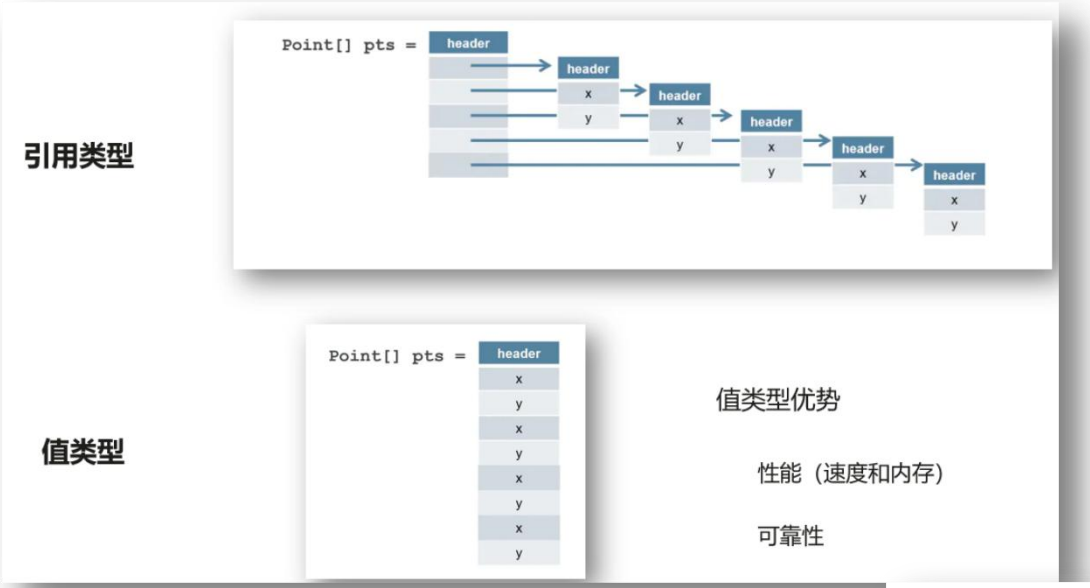
Stop-the-world GC　　vs　　并发 GC

全栈优化

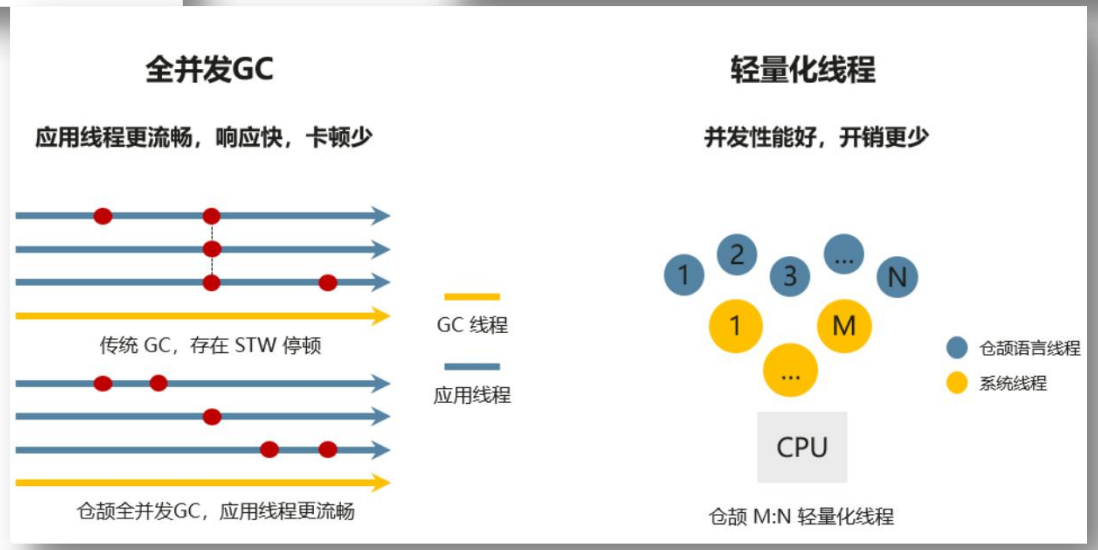
前后端编译优化



值类型



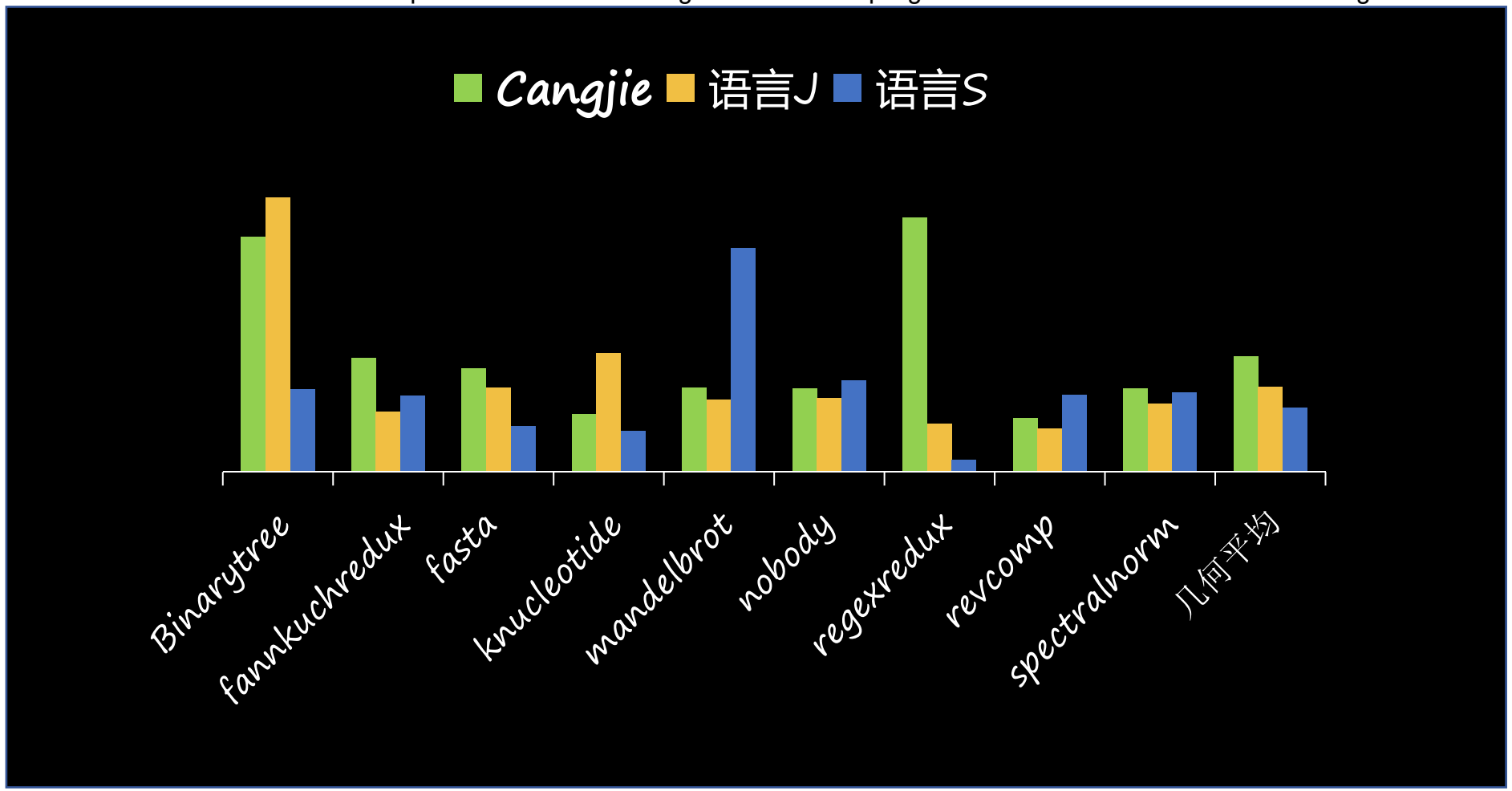
全并发GC、轻量化线程



性能

Benchmarks Game测试（越高越好）

https://benchmarksgame-team.pages.debian.net/benchmarksgame/index.html



跨语言互操作（生态扩展）

仓颉调用C代码示例

```
// stdlib.h
int rand();

// stdio.h
int printf (const char *fmt,

// declare the function by `foreign` keyword, and omit `@c`
foreign func rand(): Int32
foreign func printf(fmt: CString, ...): Int32

main() {
    // call this function by `unsafe` block
    let r = unsafe { rand() }
    println("random number ${r}")

    var fmt = CString("Hello, No.%d\n")
    unsafe { printf(fmt, 1) }
    fmt.free()
}
```

foreign 修饰函数声明
代表该函数为外部函数

使用unsafe
直接调用

仓颉与C基础类型映射

Cangjie Type	C type	Size
Unit	void	0
Bool	bool	1
Int8	int8_t	1
UInt8	uint8_t	1
Int16	int16_t	2
UInt16	uint16_t	2
Int32	int32_t	4
UInt32	uint32_t	4
Int64	int64_t	8
UInt64	uint64_t	8
IntNative	-	platform dependent
UIntNative	size_t	platform dependent
Float32	float	4
Float64	double	8

ArkTS-仓颉混合开发：跨语言互操作轻松复用软件资产

```
// arkTS 侧声明 Cangjie 侧可调用函数
interface a {
  initJsContext(ctx: Context): null,
  // 注册 arkTS 函数供 Cangjie 侧调用的工具 函数
  regJsFunc4Cangjie(name: string, val: Function): null,
}

// 加载 Cangjie 构建的库文件
let cjlib = requireCJLib("libentry_default.so") as a
// arkTS 侧调用 Cangjie 提供的函数
cjlib?.initContext(getContext(this))
// 注册 arkTS 侧函数 getUA 到 Cangjie 侧
cjlib?.regJsFunc4Cangjie(`getUA`, getUA)

function getUA(): string {
  return `hello world`;
}
```

ArkTS调用仓颉

```
// Cangjie 侧注册 arkTS 可调用的函数
static init() {
  JSModule.registerFunc("initJsContext", { context => context.function(initJsContext) })
  JSModule.registerFunc("regJsFunc4Cangjie", { context => context.function(regJsFunc4Cangjie) })
}
// Cangjie 侧函数
var jsCtx: ?JSContext = None
static public func initJsContext(context: JSContext, callInfo: JSCallInfo): JSValue {
    jsCtx = context
    context.null().toJSValue()
}
// 注册 arkTS 函数到 Cangjie 侧
// 保存在函数表中: Cangjie侧提供 JSFunction, JSValue 等类型
var jsFuncs = HashMap<String, JSFunction>()
static public func regJsFunc4Cangjie(context: JSContext,
callInfo: JSCallInfo): JSValue {
  let name: String = callInfo[0].asString(context).toString()
  let fn: JSFunction = callInfo[1].asFunction(context)
  jsFuncs.put(name, fn)
  context.null().toJSValue()
}
// Cangjie 侧调用 arkTS 的函数
let res = jsFuncs.get("getUA")?.call([])
let cjStr: String = res?.toString(jsCtx.getOrThrow())
```

仓颉调用ArkTS

材料来自李可《中国工商银行鸿蒙应用仓颉语言开发实践》

仓颉和C程序互调用

仓颉和ArkTS程序互调用

元编程支持面向领域的“声明式”开发

声明式UI

声明式开发模式更好的处理**复杂状态管理**，提升开发效率

```
@MainEntry
@Component
class MyView {
    @State var message: String = "Hello World"
    func render () {
        Flex {
            Text(this.message)
            .backgroundImage(src: @r(app.media.icon))
            .borderWidth(1)
            .backgroundImageSize(width: 50.px, height: 50.px)
            .height(150)
            .width(150)
            .onClick {
                this.message = "Hi World"
            }
        }
    }
}
```

分布式数据对象

数据在不同设备间传递及更新逻辑复杂，通过领域抽象屏蔽跨设备数据传输复杂度

```
1  // 绘制轨迹为分布式的数据
2  @Observable[DISTRIBUTED]
3  var drawTrace: List<Segment>
4
5  // 当drawTrace发生变化时，
6  // 会自动触发Reaction执行
7  @Reaction
8  func draw() {
9      for(segment: drawTrace) {
10         drawSegment(segment)
11     }
12 }
```

数据库

开发者聚焦于业务逻辑的表达，简单易用使用数据库对象与事务操作

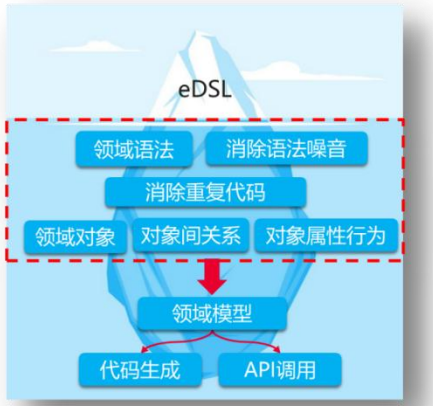
```
func postCreateAccount(account: Account)
{ // Account元数据生成类
    if (account.balance > 100000.0) {
        let customer = @OQL ( // 内嵌OQL
(eDSL)
            SELECT * FROM Customer WHERE
id = account.customerId
        )
        customer.vip = true
        update(customer) // 平台能力封装
（实体引擎eDSL）
    }
}
```

并发框架

在并发框架内，方便地定义出并发安全类，并且不需要显示地使用互斥锁

```
@Shareable
class Account {
    private var money = 0
    init(m: Int64) {
        money = m
    }
    public deposit(amount: Int64): Bool {
        money += amount
    }
    ....
}
```

仓颉的eDSL“工具箱”

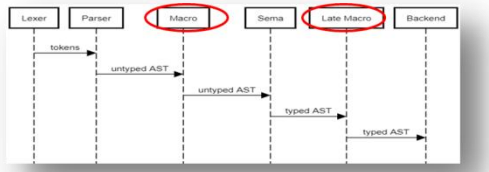


仓颉语言能力

- OO
- 类型扩展
- 链式调用
- 命名参数/默认参数
- 操作符重载
- 属性机制
- Collection literals
- 宏
- 注解
- 反射
- 类型推导
- 语法“糖”

多阶段过程宏，实现表达能力和实现难度的平衡：

宏调用可以发生在不同的阶段：
1) token stream；
2) 无类型信息的AST；亦或
3) 带类型信息的AST



模板宏：对指定模式的输入代码做变换，更加简洁直观

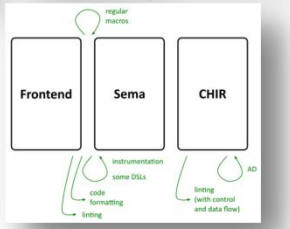
模板宏定义：

```
template macro unless {
    template(cond: Expr, block: Block) {
        @unless (cond) block
            =>
        if (! cond) block
    }
}
```

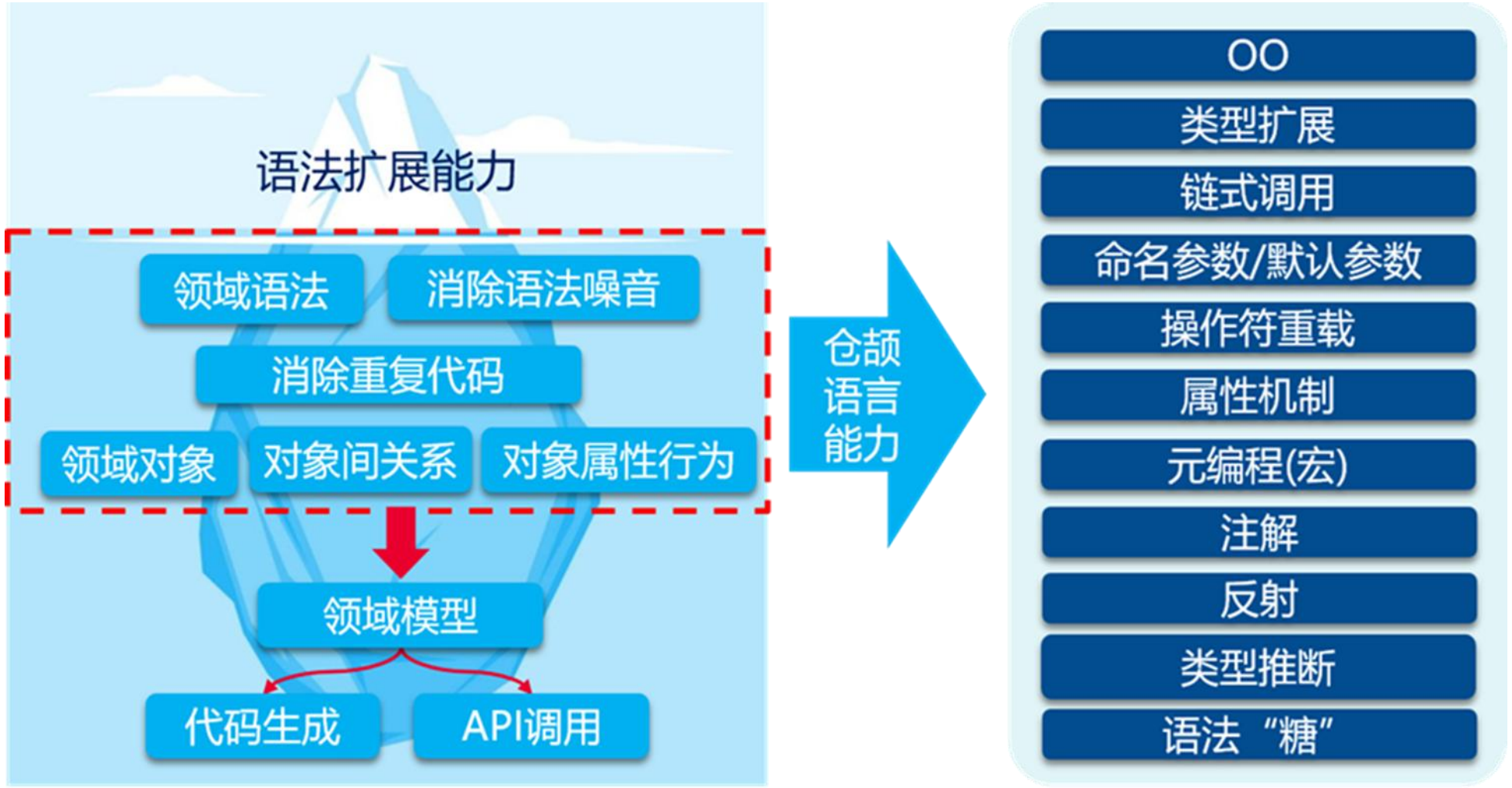
模板宏展开：

```
// ...
@unless (correctAnswer()) {
    println("nope, sorry. try again")
}
```

可定制编译流程：编译插件化，支持定制化的代码变换（例如代码插桩、代码格式化等）



仓颉的eDSL工具箱


语法扩展能力
领域语法
消除语法噪音
消除重复代码
领域对象
对象间关系
对象属性行为
领域模型
代码生成
API调用
仓颉语言能力
OO
类型扩展
链式调用
命名参数/默认参数
操作符重载
属性机制
元编程(宏)
注解
反射
类型推断
语法“糖”

尾随lambda和eDSL应用

尾随lambda 定义组件

命名参数 和 **链式调用**
定义组件属性




```
1 @Compoment
2 record Counter {
3     @State var cnt = 0
4     func build() {
5         Column {
6             Text(content: "Click $(cnt) times")
7                 .color(Color.Red)
8             Button {
9                 Text(content: "Click")
10                    .background(Color.Yellow)
11            }.onClick {
12                cnt = cnt + 1
13            }
14        }
15    }
16 }
```

类型扩展和属性

```
var bookBorrowDate = 2.days.ago
var bookExchangeDate = 3.days.later
```

- **类型扩展**：采用“extend”关键字，为类型添加新功能，并且不破坏原有类型封装性；
- **属性**：采用“prop”关键字，将数据读写封装成访问函数，外部访问与普通字段无异，便于实现访问控制、数据监控、跟踪调试、数据绑定等机制；

```
// 为原生类型Int64扩展属性
extend Int64 {
    // 属性定义
    prop var days: Int64 {
        get() { this }
        set(value) {}
    }

    prop var ago: Int64 {
        get() { DateTime.Now().Day() - this }
        set(value) {}
    }

    prop var later: Int64 {
        get() { DateTime.Now().Day() + this }
        set(value) {}
    }
}
```

过程宏和模板宏（部分能力构建中）

过程宏：一段仓颉代码，对输入代码的token stream/AST做变换

宏调用/宏展开：

```
1  @FindType
2  var e1: Employee = Employee("Fred Johnson")
3  // getting the type info of `x`: easy, it's right there
4
5  @FindType
6  var e2 = Employee("Bob Houston")
7  // getting the type info of `x`: hard, requires type analysis
```

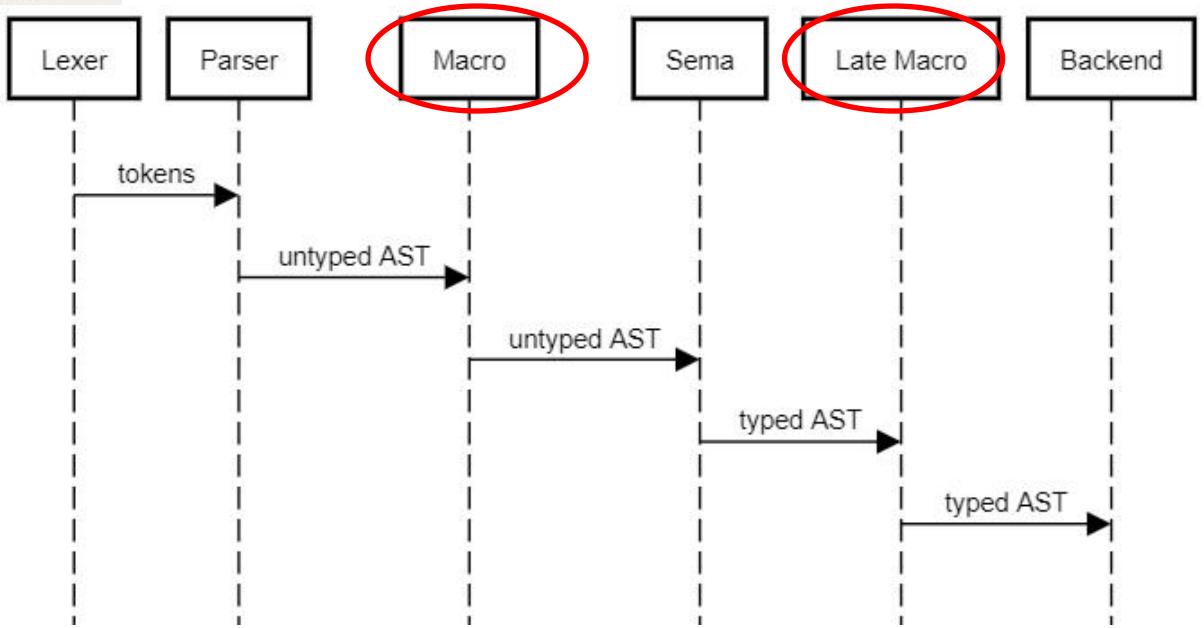
宏定义：

```
macro FindType( varDecl ) {
    …
    …
}
```

多阶段过程宏：

宏调用可以发生在不同的阶段，决定了宏输入的格式是

1) token stream；
2) 无类型信息的AST；亦或
3) 带类型信息的AST



过程宏和模板宏（部分能力构建中）

模板宏：对指定模式的输入代码做变换

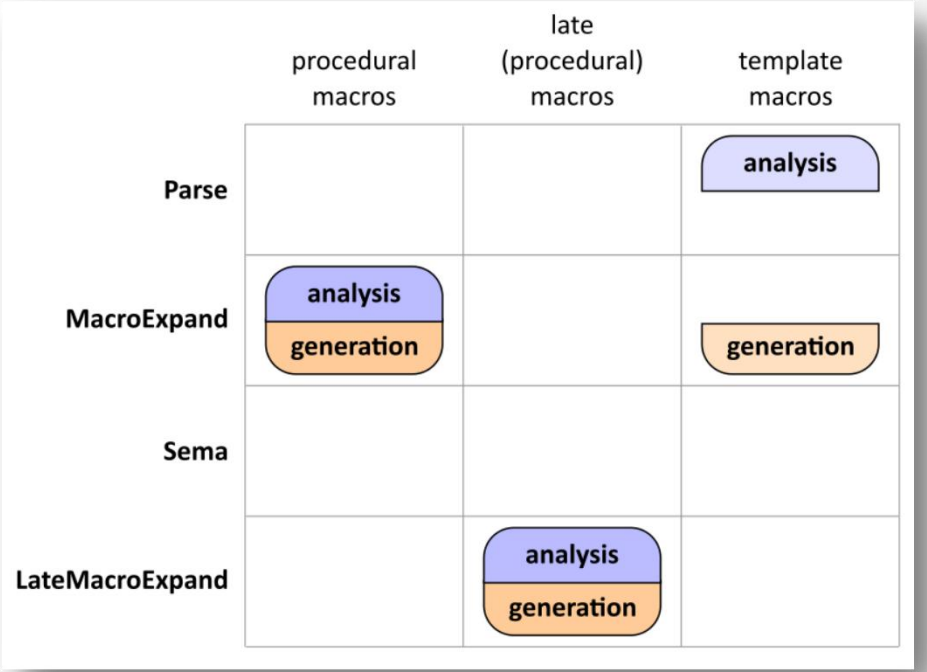
宏定义：

```
template macro unless {
    template(cond: Expr, block: Block) {
        @unless (cond) block
            =>
        if (! cond) block
    }
}
```

相比过程宏能力受限，但更容易定义和使用

宏展开：

```
// ...
@unless (correctAnswer()) {
    println("nope, sorry. try again")
}
```



报告提纲

- 仓颉语言背景介绍
- 设计理念和技术特性
- 大模型下的编程语言思考
- 未来方向和规划

大模型下的应用开发

大模型作为运行时服务

As is:

```
func convert(md: String): String {
    let md = loadFile("a.md")
    let html = convert(md)
    saveFile("a.html", html)
}
```

To be:

```
func convert(md: String): String {

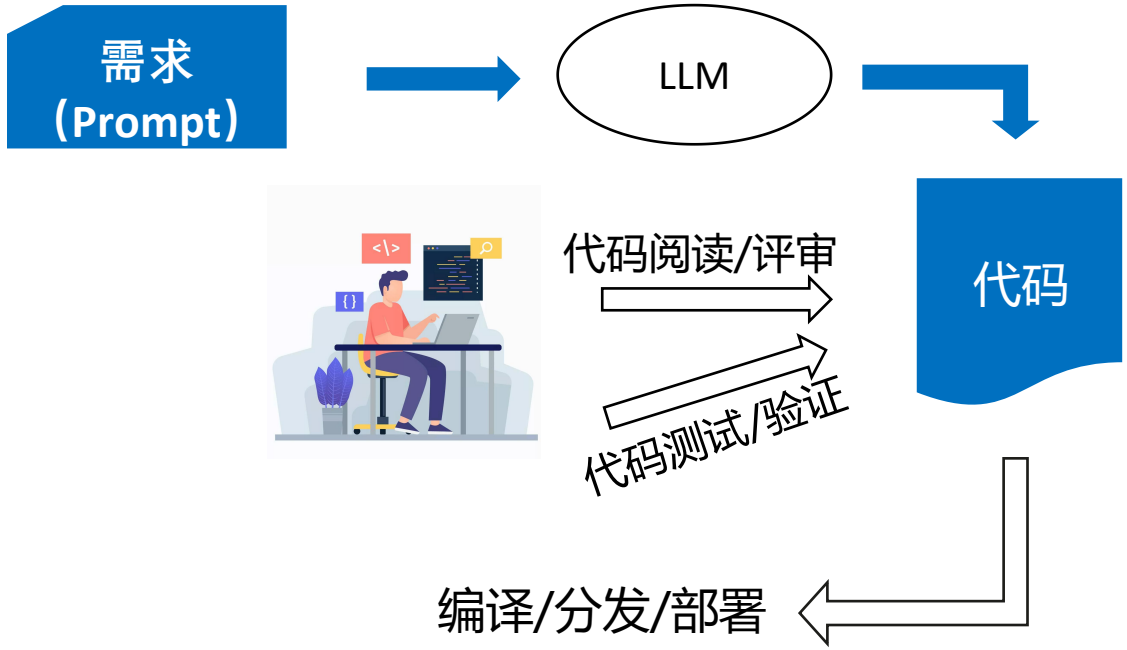
  "请把 markdown 格式的${md}
  转换为 html 格式"

}
```

LLM → 结果

关键挑战：需求的精确描述（Prompt），与大模型的交互，服务的编排，结果的校验等

大模型辅助生成代码



关键挑战：提升生成能力，代码更易读（更加声明式，Very High-Level PL），更易测试/验证

Agent原生智能应用开发（构建中）

内嵌AgentDSL的编程框架

自然语言&编程语言有机融合

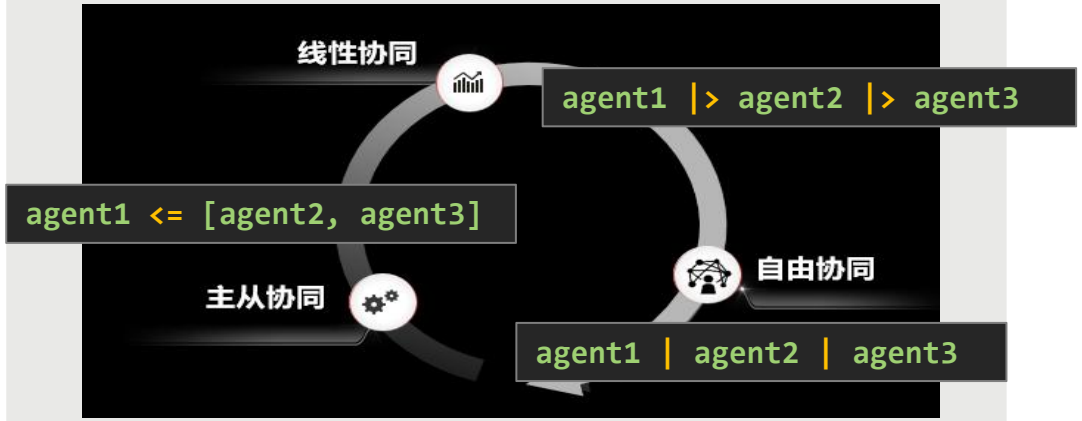
```
@agent class Planner {
  @prompt[pattern=APE] (
    action: "帮助用户制定旅行路线",
    purpose: "让用户在计划时间内多参观景点并得到充分休息",
    expectation: “生成一条合理的旅游景点路线，包括时间、景点、通勤等信息"
    )
}
```

仓颉 AgentDSL

模型聚合 | 多Agent协同 |高效检索RAG | 智能工具链

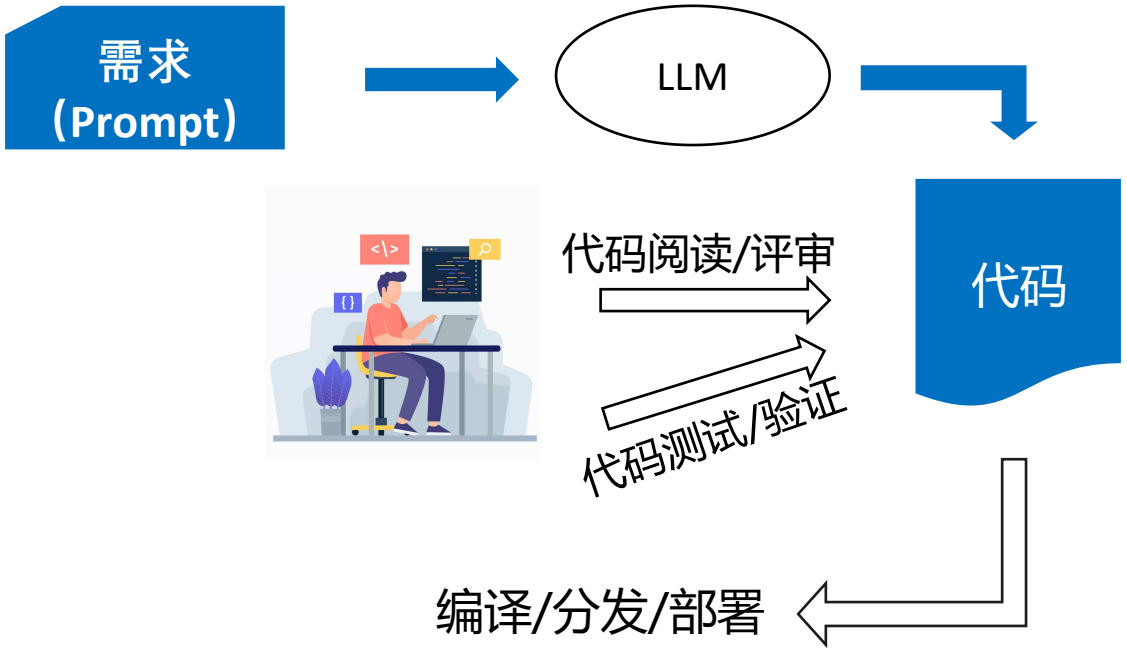
多Agent协同

简化符号表达，模式自由组合



基于仓颉的eDSL能力

对代码生成友好的编程语言？



关键挑战：提升生成能力，代码更易读（更加声明式，Very High-Level PL），更易测试/验证

IDE

代码生成

对经典软件工程实践的更好支持：
代码评审、测试、验证等

编程语言：

代码更易阅读、评审、验证、测试等

更加声明式、抽象级别更高的语言

➡ **面向特定领域，构建DSL**

Self-verifying/
Self-validating lang.

➡ **契约编程**

报告提纲

- 仓颉语言背景介绍
- 设计理念和技术特性
- 大模型下的编程语言思考
- 未来方向和规划

语言设计中的一些两难选择和挑战

协程：有栈 （Java，Go）**vs.** 无栈（C#, C++, Rust, Kotlin, Swift）

→ Effect handler + effect typing

异常处理：Checked exception **vs.** unchecked exception **vs.** Results **vs.** error codes → Effect typing

并发：共享内存 **vs.** 消息发送 → Ownership

资源管理：RAII → Ownership typing + Borrow checking + …

Effects

易用、简洁

编程模型



统一的语言设计

元编程、通用表达

领域表达、宏、...

Effects

异常、协程、Actor、...

优雅的类型世界

通用类型

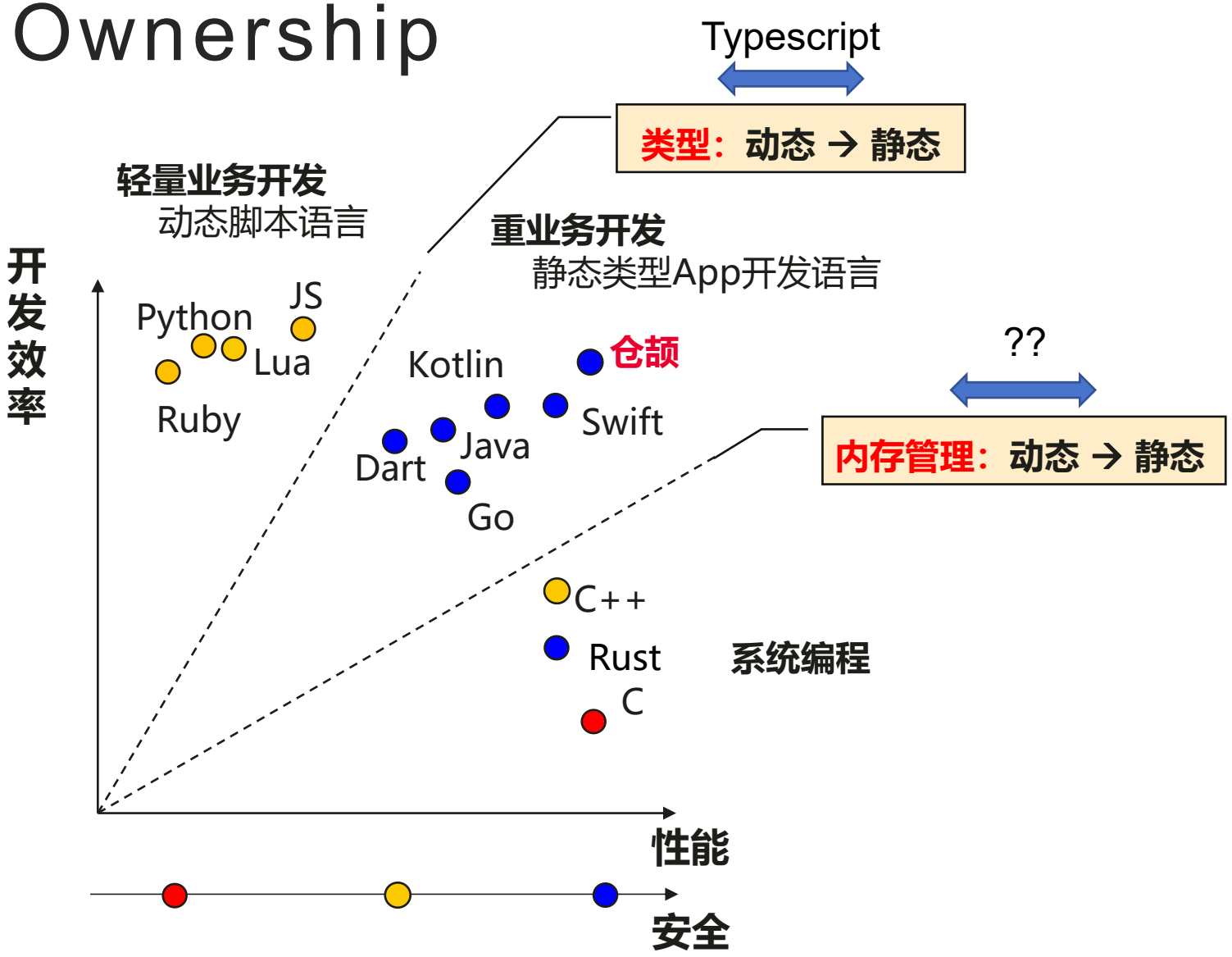
泛型、函数、类、...

Effects Types

安全、高效、多态

Effects System：轻松**定义、定制和组合**一系列编程特性，“乐高式” 软件开发

Ownership



注：此图仅为示意三类语言的划分，不追求各种语言在坐标中位置的准确性

Ownership：

安全资源管理

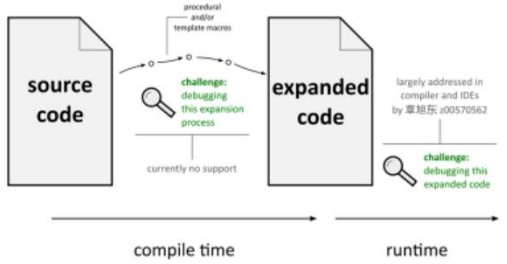
并发安全

GC的补充

更加丰富的宏和元编程机制

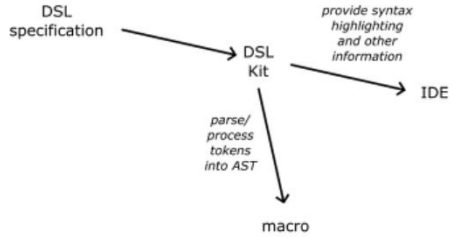
Macro debugging

- People may write macros that work right away, but probably also many will need to be debugged.
- The problem is visibility – introspecting the macro expansion process.
- **Technical challenge**: IDEs can already show the fully expanded code. What's the next step?



DSL Kit

- An on-ramp for DSL authors
  - Easy to declaratively specify a BNF grammar (instead of hand-coding a parser)
  - Custom AST, static type checking
  - Easy integration with IDEs
- A "DSL compiler toolkit"
  - Ease of use, meaningful error messages, and a default set of tools



User Compile Passes (UCP)

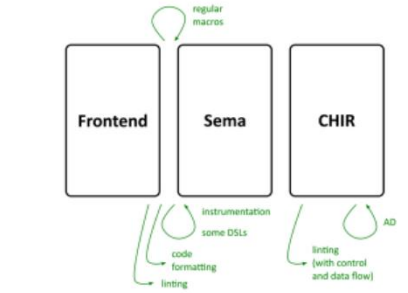
- Macros are powerful, but *local*.
- Some use cases require whole-program transformation:
  - Instrumentation (profiling/coverage/fuzzing)
  - User-defined linting
  - DSL-specific checking/transformation
- Compile passes operate on the code from the *outside*, as tools.
- Find a declarative design, like template macros

```
$ cjc file.cj --with-pass=linter.cj
```

```
// file: linter.cj
from compilepass import after_parse.*

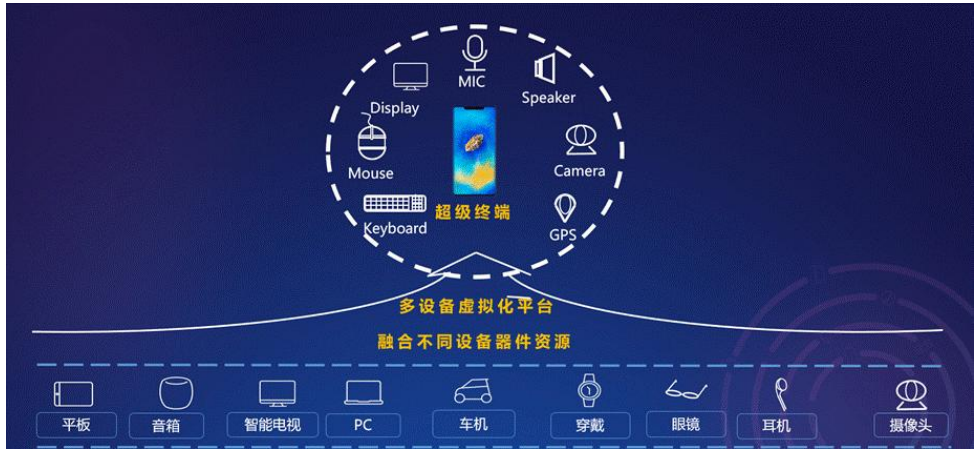
func onLetOrVar(ast: VarDecl) {
    // linting for variable declarations
}

func afterPass() {
    // print out any linter errors
    compiler.exit()
}
```

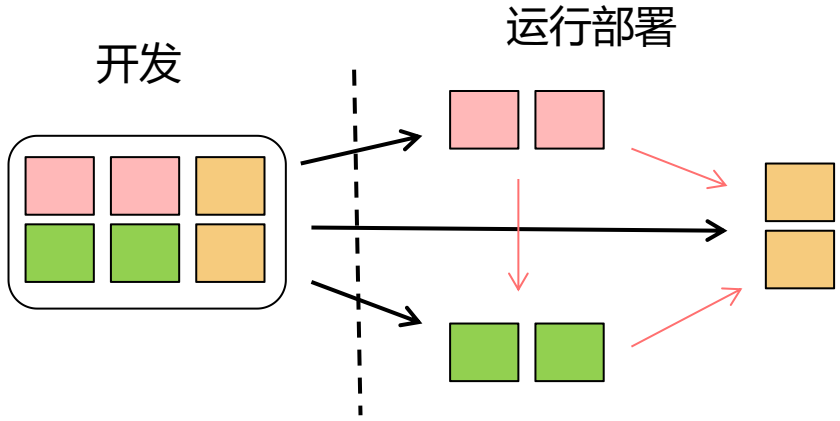


Actor统一并发和分布式编程

多设备协同

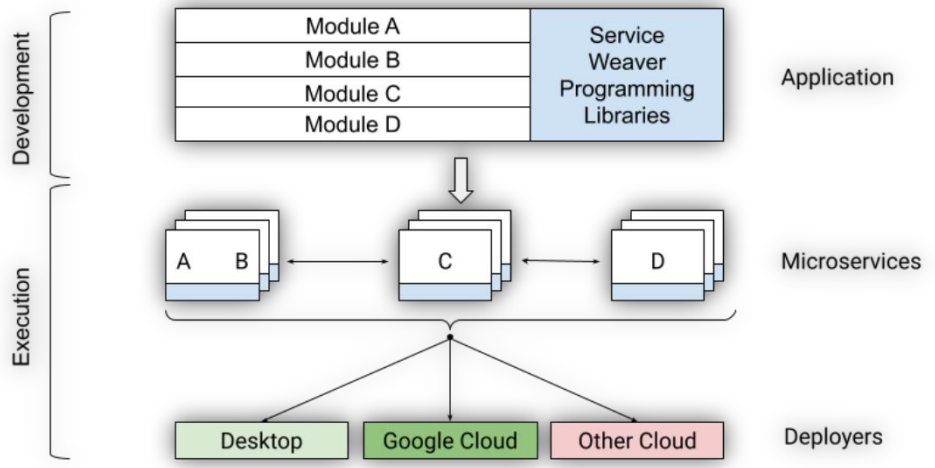


需求：单体程序开发，多设备部署？



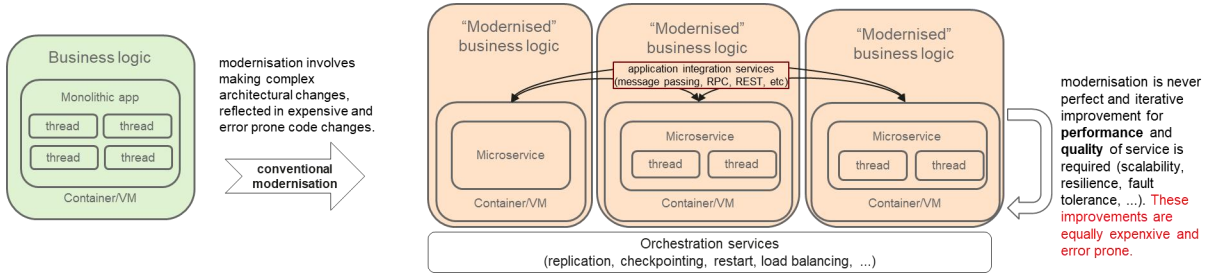
端云协同/云原生

需求：简化开发和部署



Google Service Weaver

需求：旧应用改造



工具、三方库、开发框架

工具：AI辅助开发、性能分析、代码扫描、版本管理、代码扫描、中心仓 …

框架：UI框架、AI框架、后端开发、数据库、分布式 …

生态互通：跨语言互操作、代码转换（X to Cangjie） …

总结

仓颉 Cangjie

仓颉：全场景智能化应用编程语言

繁荣HarmonyOS应用生态

拥抱大模型时代的新应用开发范式

- Agent DSL和框架
- 更好的测试/验证能力

无限的成长空间…



CSDN Boolan

SDCon 2024

全球软件研发技术大会

Software Development Conference

谢谢

